

progetto asd

PIETRO FUBINI

May 2026

1 Come gestire i dati

come deve essere il grafo finale:

1. lista hasciata dei nodi
2. oogni nodo e una n-upla (nome nodo,hascing function, lista hasciata di adiacenza) la lista hasciata di adiacenza è

ogni nodo(formato dal nome nodo e la fuunzione di hascing universale) ha la lista di adiacenza hasciata con hascing universale
la lista di adiacenze è formmata da coppie, con il vicino e il peso

i nodi sono a loro volta hasciati in una tabella per cercarli bene

1.1 input file

- input: prende il file **all-paths** con un cin
- grafo non orientato $G=(n,e)$, con edge pesati
- funzionalita:
 1. crea tutti i vari archi pian piano che li legge usando la funzione aggiungi arco
 2. chiude il file

1.2 funzioni utili

1.2.1 albero puntato

- input: nodo partenza e arrivo, cap sulla lunghezza
- processo: prende un nodo e scorre tutta la sua lista di adiacenza, aggiungendo tutti i nodi con meno della distanza max, ogni nodo aggiunto è inserito nella terna:
 1. nome nodo

2. nodo babbo
3. insieme hasciato di nodi già percorsi

continua questo processo se trova il nodo di arrivo si ferma, se non lo trova usa cancella nodo in un albero dal nodo iniziale di questo ramo

- output: un albero con tutti i percorsi banalmente ogni ramo fogliato è un percorso diverso

1.2.2 cancella nodo in un albero

- input: nodo
- procedimento: passa la funzione a tutti i suoi figli e poi quando torna a lui elimina il nodo
- output: albero senza il nodo e tutti i suoi figli

1.2.3 lista adiacenza

prende in input un nodo e ritorna un vettore di coppie con la sua lista di adiacenza con i pesi

1.2.4 aggiungi nodo

aggiunge il nodo, randomizzando una hash universale, prende in input un int (i vari nodi sono degli int) e per l'albero dereferenziato

1.2.5 aggiungi arco

input: nodo 1 e nodo 2

processo: va in nodo uno e cerca nella lista di adiacenza con la funzione di hashing il nodo 2 se non ce lo aggiunge con il peso 1 altrimenti aumenta di 1 il peso fa lo al contrario, rehasha se necessario

in teoria è $O(1)$

output: è aggiunto in entrambe delle liste di adiacenza in più uno su quell'arco

2 da .all-paths a struttura di grafo

prende il testo linea linea e spara il grafo. ogni passaggio prende 2 rami, checka se i vertici esistono già e si aggiunge o somma uno alla lunghezza del ramo altrimenti aggiunge i vertici e aggiunge 1 alla lunghezza del ramo. chiaramente il ramo non esistente conta come 0 anche se non lo creo per questioni di memoria

è importante ridia anche la lunghezza massima di un arco

3 funzioni utili

3.1 conta foglie

input: nodo

processo: ricorsivo:

- passo base: il nodo non ha figli ritorna 1;
- passo induttivo il nodo ha figli, ritorna la somma del risultato dei figli

output: numero di foglie

costo: visita ogni foglia 1 volta e su ogni foglia ci la funzione ci mette $O(1)$
quindi il costo è $O(n)$

4 minimo percorso tra 2

tipologia algoritmo: Greedy, programmazione dinamica.

input:: grafo pesato, nodi da collegare

output:: distanza tra i due

algoritmo: Prende il nodo e (controlla se ce l'ultimo nodo) lascia i suoi vicini con la funzione di hashing che dà la distanza dal vertice di partenza a quello valutato, e poi lascia universalmente in ogni set di lunghezza, e poi fa la ricorsione (sul primo della lista relativa al percorso più leggero), passandosi ogni volta che fa il numero più alto preso.

ce infine un'ultima lista lasciata dei nodi già visitati che viene tenuta per evitare di riesplorare più volte il lo stesso nodo, tanto per lo stesso ragionamento di di sinistra non ce un percorso più vicino che ci arriva

i casi doppi: per evitare questi casi l'algoritmo, prima di aggiungere un nodo nella lista di quelli da controllare guarda, innanzitutto se è già stato controllato, se si passa al prossimo, se non è ancora stato controllato guarda se è già in lista per lunghezze minori, e per sicurezza anche in quelle maggiori, per evitare di riempire la struttura di spazzatura

costo computazionale: passa da ogni nodo al massimo 1 volta (visto che prima di aprire 1 nodo conta se è già stato aperto). Quando è nel nodo:

- guarda tutti i vicini, costo $O(n)$
- in ogni vicino controlla se ce nella lista dei già visitati costo $O(1)$, se no allora controlla se sopra o sotto